An Automated Pipeline for Offline Deployment of Custom Computer-Vision Models on iOS

Anh Pham
Department of Electrical and Computer Engineering
Rice University, Houston, TX, USA
ELEC 594: Project Course

***Abstract*—Cloud-based computer vision introduces network latency and exposes sensitive imagery, while deploying custom deep-learning models to iOS remains hindered by conversion friction and tight on-device resource budgets. We propose an automated pipeline and native iOS application framework that translates arbitrary vision models to Core ML, compresses them through mixed-precision quantization, structured pruning, and retraining, and deploys them for fully offline, real-time inference. We report partial results: a custom YOLO detector was deployed to a physical iPhone and runs at 30.3 frames per second (13.1 ms per frame), and post-training INT8 quantization reduced model size by 74% (9.9 to 2.6 MB) and inference time by 3.2× (16.5 to 5.2 ms) while preserving precision and recall on 128 COCO images. We next target fully on-device detection of bead-labeled CD4+ T cells for point-of-care diagnostics.**

***Keywords*—edge AI, Core ML, model quantization, object detection, on-device inference, point-of-care diagnostics**

I. Introduction

Modern computer-vision applications increasingly rely on remote servers for inference. This cloud dependency introduces two persistent problems. First, every frame must traverse the network, adding latency and failing entirely when connectivity is unavailable. Second, transmitting raw imagery—faces, private environments, or clinical samples—off the device creates serious privacy exposure. For many emerging applications, including assistive technology, field robotics, and point-of-care medical diagnostics, the ideal system performs inference locally, in real time, with no data leaving the device.

Running custom models on a smartphone, however, is not straightforward. Apple's Core ML runtime executes models efficiently on the iPhone's Neural Engine [1], but moving a trained network into that runtime requires architecture conversion, Swift and Vision integration, and code signing, all under strict compute, memory, and battery budgets. Existing tooling addresses isolated slices of this problem: Core ML converters handle one-off format translation, and compression methods such as integer quantization [2] and pruning [3] are typically applied manually, per model. No unified, automated framework takes an arbitrary custom architecture and produces a compressed, deployed, offline iOS application.

This project proposes such a framework. Our objective is an end-to-end pipeline that (i) translates an arbitrary vision model to Core ML, (ii) compresses it automatically while preserving accuracy, and (iii) deploys it into a reusable native iOS application for real-time, offline inference. This report presents the system design and partial results from the first phase: a working on-device deployment and a measured, automated quantization stage.

Source, figures, and build files: https://github.com/ap121Rice/ELEC594Fall2026

II. Methods

A. Pipeline Overview

The framework is organized as the four-stage pipeline shown in Fig. 1. A trained model enters the *translate* stage, which converts it to the Core ML format. The *compress* stage applies mixed-precision quantization, structured pruning, and a short retraining pass to recover any lost accuracy. The *deploy* stage embeds the resulting model in a native iOS application, and the *run* stage performs inference entirely on the device. Because the application loads models through a generic interface, swapping in a new architecture does not require changes to the surrounding app code.

B. Native iOS Application

The application is built in Swift and SwiftUI using an MVVM architecture, with inference isolated behind a model-agnostic module that bridges the camera feed to Core ML through Apple's Vision framework. We configured the Xcode build-and-signing toolchain to deploy to a physical iPhone, which is required to measure realistic on-device throughput and latency. The YOLO family [4] is used as the reference detector because of its strong accuracy-to-latency trade-off; the current build uses a custom `yolo26n` model exported from the Ultralytics framework [5].

C. Model Compression and Evaluation

To establish the compression stage, we built an automated quantize-and-evaluate script that converts a model from 16-bit floating point (FP16) to 8-bit integer (INT8) precision [2] and benchmarks the result. We evaluate three quantities against the uncompressed baseline: detection accuracy (precision and recall), on-disk model size, and per-image inference time, measured on a fixed set of 128 COCO validation images [6]. This harness is model-independent, so it will apply unchanged to future custom models.

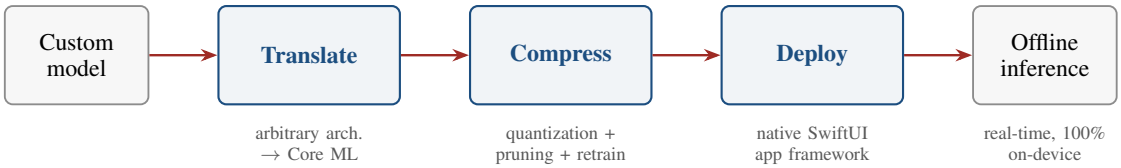


Fig. 1. The proposed pipeline. A custom vision model is translated to Core ML, automatically compressed, and deployed into a reusable native iOS application for real-time, fully offline inference. Each stage is a scriptable, reusable step rather than a one-off manual port.

TABLE I. EFFECT OF INT8 QUANTIZATION (128 COCO IMAGES)

Metric	FP16	INT8	Change
Precision	0.752	0.743	-1.2%
Recall	0.534	0.540	$+1.1\%$
Model size (MB)	9.9	2.6	-74%
Inference (ms)	16.5	5.2	$3.2\times$ faster

D. Target Application

Our target application is on-device detection of CD4+ T cells for point-of-care immune monitoring. Dixit *et al.* [7] demonstrate an AI-enabled microfluidic cytometer in which a YOLO detector counts antibody-bead-labeled CD4+ T cells in microscope video of whole blood, with accuracy comparable to flow cytometry. In that work inference runs on a cloud GPU; porting the detector through our pipeline onto an iPhone would remove the cloud dependency and enable the portable, low-resource operation the authors envision.

III. RESULTS (PARTIAL)

A. On-Device Deployment

The custom `yolo26n` model was successfully deployed to a physical iPhone within our SwiftUI application and performs live object detection in real time, sustaining 30.3 frames per second at 13.1 ms per frame (Fig. 2). Because the model was swapped in without modifying the application, this result validates the model-agnostic loader and confirms that the hardest integration steps—physical-device deployment and custom-model loading—function end to end.

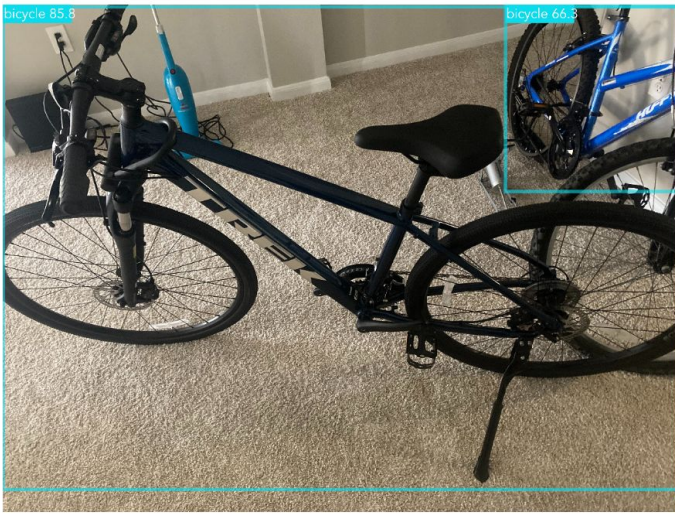


Fig. 2. Custom `yolo26n` detector running fully on-device in our iOS application, with per-object confidence scores. The live camera build sustains 30.3 FPS (13.1 ms per frame).

B. Automated Quantization

Table I and Fig. 3 report the measured effect of INT8 quantization. Model size fell by 74% (9.9 to 2.6 MB) and per-image inference time fell by $3.2\times$ (16.5 to 5.2 ms), while precision and recall were essentially unchanged (precision $0.752 \rightarrow 0.743$; recall $0.534 \rightarrow 0.540$). Compression therefore yields a large efficiency gain at negligible accuracy cost on this workload.

IV. DISCUSSION (PARTIAL)

The partial results de-risk the two highest-uncertainty parts of the project. On-device deployment of a custom model is working, and the compression stage already delivers substantial, automated efficiency gains with negligible accuracy loss—evidence that the project's end-of-semester targets (a $10\times$ size reduction, $\leq$18 ms per-frame inference, and $\geq$98.5% of baseline accuracy) are realistic. Several limitations remain. The quantization results were obtained on a general detector and the standard COCO set, not yet on the target cell-detection task; structured pruning and retraining are specified but not yet integrated into the automated stage; and the four stages are presently run in sequence rather than as a single scripted flow.

The next phase addresses these gaps in order: integrate pruning and retraining into the compression stage; train and convert a CD4+ T-cell detector following the dataset and evaluation protocol of Dixit *et al.* [7], targeting mean average precision and cell-count agreement comparable to the published cloud model; and deploy that detector on the iPhone, benchmarking on-device latency and accuracy against the cloud baseline. Success would demonstrate a complete path from an arbitrary custom model to a private, offline, point-of-care diagnostic application.

ACKNOWLEDGEMENTS

The author thanks the instructor and teaching staff of ELEC 594 at Rice University for their guidance, and acknowledges the open-source Ultralytics and Core ML tooling used in this work.

CREDIT AUTHOR STATEMENT

Anh Pham: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Visualization, Writing – original draft, Writing – review & editing. As a single-author (team-of-one) project, all contributor roles were carried out by the author.

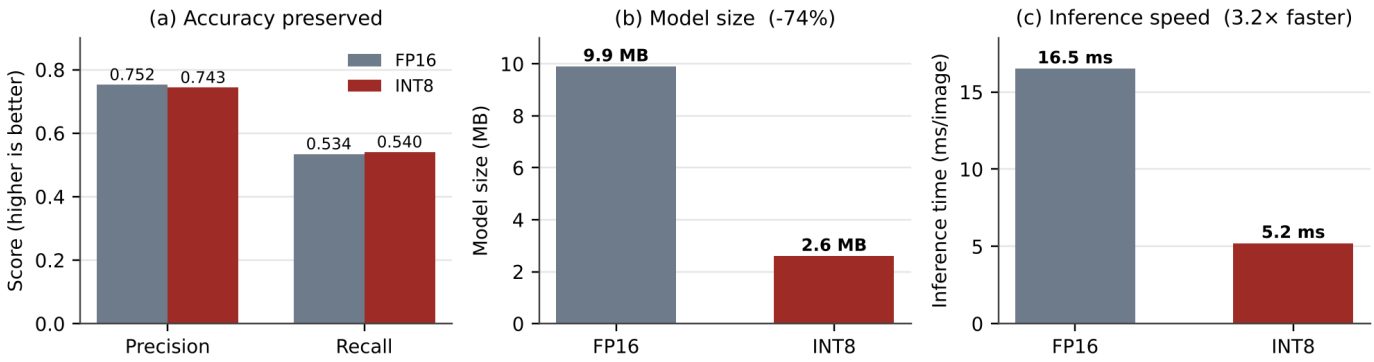


Fig. 3. Measured impact of FP16 → INT8 quantization on a reference detector, evaluated on 128 COCO images: (a) precision and recall are preserved, (b) model size drops 74%, and (c) inference is 3.2× faster. The same automated harness will be applied to future custom models.

REFERENCES

[1] Apple Inc., "Core ML: Integrate machine learning models into your app," https://developer.apple.com/documentation/coreml, accessed: Oct. 2026.

[2] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 2704–2713.

[3] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2016.

[4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 779–788.

[5] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLO," https://github.com/ultralytics/ultralytics, 2023.

[6] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2014, pp. 740–755.

[7] D. D. Dixit, T. P. Graf, K. J. McHugh, and P. B. Lillehoj, "Artificial intelligence-enabled microfluidic cytometer using gravity-driven slug flow for rapid CD4+ T cell quantification in whole blood," *Microsystems & Nanoengineering*, vol. 11, no. 1, p. 36, 2025.